

Uniwersytet Wrocławski

Generatory Liczb Pseudolosowych

Autorzy: Łukasz Stodółka, Łukasz Besuch, Marcin Rypula

Wrocław, 2026

Contents

1	Jakis rozdział	2
1.1	Coś tam	2
1.2	Model teoretyczny i wyniki	2
1.3	Model matematyczny	2
1.4	Zastosowanie referencji	2
2	Lorem	4
2.1	Ipsum	4
2.2	Dolor	4
2.3	Sit	4
3	Linear Feedback Shift Register (LFSR)	5
3.1	Maximum Length Sequence:	5
3.2	The Mathematics of LFSR	5
4	Lagged Fibonacci Generator (LFG)	6
4.1	The Standard Fibonacci Sequence	6
4.2	Refining Fibonacci Sequence	6
4.3	Different versions of LFG:	7
4.4	The Mechanism in Practice	7
4.5	Key Characteristics of LFG	7

1 Jakis rozdzial

1.1 Coś tam

Listing 1: Plik ./codes/example.cpp

```
1 #include <iostream>
2
3 int main() {
4     std::cout<<"Hello World!";
5 }
```

`std::vector`, $\mathbb{R}^n$.

1.2 Model teoretyczny i wyniki

$\mathbb{E}[X]$ nad zbiorem $\mathbb{Z}$.

Twierdzenie 1.1. *Jakieś $n \in \mathbb{N}$ coś tam blabla bla, istnieje rys. 1.*

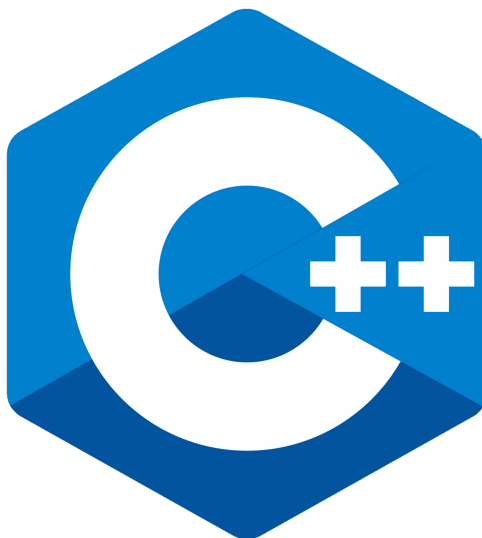


Figure 1: Przykładowe

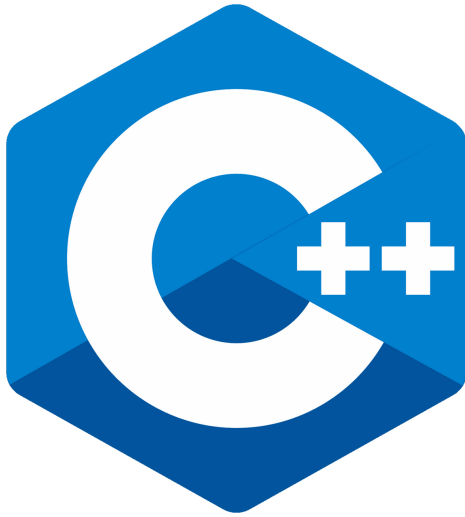
1.3 Model matematyczny

1.4 Zastosowanie referencji

- Odwołanie do rysunku: rys. 1.
- Odwołanie do sekcji: sekcja 1.3 .

- Odwołanie do literatury: [1].

Można dwa obrazki:



(a) pierwszy obrazek



(b) Drugi obrazek

Figure 2: Coś tam blabla bla

2 Lorem

Lorem ipsum dolor sit amet aa, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi.

2.1 Ipsum

Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

2.2 Dolor

Ut in risus volutpat libero pharetra tempor. Cras vestibulum bibendum augue. Praesent egestas leo in pede. Praesent blandit odio eu enim. Pellentesque sed dui ut augue blandit sodales. [1]

2.3 Sit

Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam nibh. Mauris ac mauris sed pede pellentesque fermentum. Maecenas adipiscing ante non diam sodales hendrerit.

3 Linear Feedback Shift Register (LFSR)

It is incredibly fast, operates fundamentally on binary numbers (0s and 1s), and is widely used in digital electronics and simple cryptography. LFSR consists of:

1. **Shift Register:** A device that shifts contents into adjacent positions in register or out of register if position is on the end. The positions where the content was is left empty if there is no content that that could be shifted there.
2. **Output Bit:** The bit at the end that is moved out of a shift register.
3. **Feedback Function:** A function that takes bits from certain positions "**taps**", combines them in some sort of function and the result is fed back into the register's input bit. Bits are collected before the shift.

3.1 Maximum Length Sequence:

An n-bit LFSR has 2^n possible states. However, because a state of all zeros will infinitely produce zeros, the maximum possible period for an LFSR sequence is $2^n - 1$.

3.2 The Mathematics of LFSR

In mathematical terms, the new bit b_n is calculated recursively based on the previous bits. If we have a register of length k , and the taps are defined by coefficients c_i (where c_i is 1 if there is a tap, and 0 if there isn't), the formula looks like this:

$$b_n = (c_1b_{n-1} + c_2b_{n-2} + \dots + c_kb_{n-k}) \pmod{2}$$

(Note: In binary math, addition modulo 2 is exactly the same thing as the XOR operation).

This relationship is represented by a characteristic polynomial:

$$P(x) = 1 + c_1x + c_2x^2 + \dots + c_nx^n$$

To achieve the maximum period of $2^n - 1$, the characteristic polynomial $P(x)$ must be a primitive polynomial over $GF(2)$.

Example of LFSR for 4-bit number with tap sequence [3, 4] and **XOR** being the feedback function:

a	b	c (Tap)	d (Tap)	Output
1	0	0	1	1
1	1	0	0	0
0	1	1	0	0
1	0	1	1	1
0	1	0	1	1
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1
0	1	1	1	1
0	0	1	1	1
0	0	0	1	1
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
1	0	0	1	1

4 Lagged Fibonacci Generator (LFG)

4.1 The Standard Fibonacci Sequence

In a classic Fibonacci sequence, the next number is simply the sum of the two numbers immediately preceding it.

$$X_n = X_{n-1} + X_{n-2}$$

However, this approach wouldn't be good for generating randomness because it is entirely predictable, and the numbers grow infinitely large.

4.2 Refining Fibonacci Sequence

Lagged Fibonacci Generator changes the Fibonacci rule to: the new number is a combination of two previous numbers from the "lag" positions, constrained using modulo operation:

$$X_n = X_{n-j} \star X_{n-k} \pmod{m}, 0 < j < k$$

- X_{n-j} and X_{n-k} are the two previously generated numbers from the "lag" positions (j and k steps back in the sequence).
- $\star$ represents a mathematical operator.
- m is the maximum limit to keep the numbers contained.

4.3 Different versions of LFG:

1. **ALFG:** Additive Lagged Fibonacci Generator. Uses addition. If m is defined as power of 2, so $m = 2^M$, the maximal length of generated sequence is $(2^k - 1) * 2^{M-1}$.
2. **MLFG:** Multiplicative Lagged Fibonacci Generator. Uses multiplication. These require all initial seeds to be odd to maintain the maximum period. If m is defined as power of 2, so $m = 2^M$, the maximal length of generated sequence is $(2^k - 1) * 2^{M-3}$.
3. **GFSR:** Two-tap Generalized Feedback Shift Register. Uses the bitwise XOR operator. This is equivalent to an LFSR, the maximal length of generated sequence is $(2^k - 1)$.

4.4 The Mechanism in Practice

When the generator needs to create a new number, it looks at the number generated j and k steps ago, combines them using its operator, applies the modulo, and outputs the result. The oldest number in the memory is discarded, the new number is added to the recent memory, and the system moves forward. Because of LFG's characteristics, it requires k numbers to start the algorithm.

4.5 Key Characteristics of LFG

- **Massive State Space:** Because maximum length is heavily dependent on number k it allows the algorithm to have an incredibly long period before the sequence of random numbers ever repeats itself.
- **Sensitivity to Setup:** The choice of the lags (j and k) is highly critical. To achieve the maximum period, the characteristic polynomial $P(x) = x^k + x^j + 1$ must be a irreducible polynomial.

References

- [1] Donald Knuth. “The Art of Computer Programming”. In: (1997).